



Pasión por la Tecnología

Fundamentos de Gestión de Datos

Docente:
Pilar Rocío Sayán Mejía

LABORATORIO N° 04

Semana 4:

Normalización y SQL Básico



LABORATORIO DIRIGIDO N° 04	
Tema: Normalización de Bases de Datos y Consultas SQL Básicas	
DOCENTE: Pilar Rocío Sayán Mejía	CURSO: Fundamentos de Gestión de Datos

I. Elemento de la Capacidad Terminal de la Semana

Identifica violaciones a la Primera, Segunda y Tercera Forma Normal en una tabla desnormalizada, interpreta el modelo Entidad-Relación de la panadería Dulce Migaja e implementa consultas SQL básicas sobre la base de datos normalizada en SQLite, relacionando el diseño relacional con decisiones de negocio reales.

Objetivo del Laboratorio

Al finalizar este laboratorio, el estudiante será capaz de:

- Identificar violaciones a la Primera, Segunda y Tercera Forma Normal en una tabla desnormalizada.
- Comprender el proceso de normalización como solución a problemas concretos de gestión de datos.
- Interpretar un modelo Entidad-Relación, sus entidades, atributos, claves y cardinalidades.
- Leer y comprender sentencias DDL (CREATE TABLE) con restricciones PK, FK, UNIQUE y CHECK.
- Interpretar consultas SQL básicas (SELECT, INNER JOIN) aplicadas al modelo normalizado.

Nota: Esta sesión tiene como objetivo que INTERPRETES el código y el proceso de normalización. En la semana 5 practicarás la escritura autónoma.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red sin supervisión.
- Prohibida la ingesta de alimentos y bebidas dentro del laboratorio.
- Ubicar maletines y mochilas en el lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla limpias y ordenadas al finalizar.
- Todo el código se ejecuta únicamente en Google Colab mediante SQLite. No se requiere instalar software adicional.

III. Fundamento Teórico

Antes de realizar este laboratorio, el estudiante debe haber revisado el siguiente material:

- Diapositivas de la Semana 4: Normalización de Bases de Datos y SQL Básico.
- Coronel, C. & Morris, S. (2019). Database Systems: Design, Implementation & Management. Cengage. Capítulos 4 y 5.
- Documentación SQLite: <https://www.sqlite.org/docs.html>

Conceptos clave:

Tabla desnormalizada, redundancia, anomalías de actualización e inserción, 1FN (Primera Forma Normal), 2FN (Segunda Forma Normal), 3FN (Tercera Forma Normal), clave primaria (PK), clave foránea (FK), dependencia funcional, dependencia parcial, dependencia transitiva, SELECT, WHERE, ORDER BY, GROUP BY, JOIN, SQLite, pandas.

Revise el material de la semana correspondiente antes de iniciar el laboratorio: Diapositivas Semana 4 — Modelo Entidad-Relación, Cardinalidad y Normalización (1FN, 2FN, 3FN).

Conceptos clave: Tabla desnormalizada, redundancia, anomalías de actualización, 1FN, 2FN, 3FN, clave primaria (PK), clave foránea (FK), dependencia funcional, dependencia parcial, dependencia transitiva, cardinalidad, SELECT, WHERE, ORDER BY, GROUP BY, JOIN, SQLite, pandas.

IV. Normas Empleadas

No aplica norma técnica específica para este laboratorio.

V. Recursos

- Computadora con acceso a internet y navegador web.
- Cuenta de Google para acceder a Google Colab.
- Base de datos: panaderia_normalizacion.db — se descarga automáticamente desde GitHub.
- Librerías: sqlite3 (estándar Python) y pandas (preinstalada en Colab).
- URL de la base de datos:
https://raw.githubusercontent.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos/main/casos/17_panaderia_normalizacion/panaderia_normalizacion.db

VI. Metodología para el Desarrollo

- El laboratorio se desarrolla de forma **individual** en un tiempo estimado de 2 horas pedagógicas.

- Entregable: Enlace del Notebook .ipynb ejecutado con código, resultados visibles y respuestas completas a todas las preguntas.
- Subir el notebook al portafolio o repositorio indicado por la docente.

VII. Procedimiento

Caso de Negocio: La Panadería Dulce Migaja

Dulce Migaja es una pequeña cadena de panaderías artesanales con varios locales en la ciudad. Durante años, el dueño registró todas sus ventas en una sola tabla llamada `ventas_original`. En esa tabla, cada fila repetía el nombre completo del cliente, el producto, la categoría y el local donde ocurrió la venta. Cuando el dueño quería corregir un dato, tenía que buscar y actualizar cada fila manualmente, generando errores e inconsistencias. Para resolver esto, el equipo normalizó la base de datos separando la información en tablas independientes: categorías, productos, clientes, locales y ventas. Tu rol: eres el analista de datos de Dulce Migaja. Primero explorarás la tabla original para entender el problema, luego trabajarás con las tablas normalizadas.

Actividad 1: Revisión de Conceptos

Completa la siguiente tabla con tus propias palabras. No copies textualmente del material. Usa el caso Dulce Migaja para los ejemplos.

Concepto	Definición con tus palabras	Ejemplo de Dulce Migaja
Primera Forma Normal (1FN)		
Segunda Forma Normal (2FN)		
Tercera Forma Normal (3FN)		
Dependencia parcial		
Dependencia transitiva		
Clave Primaria (PK)		
Clave Foránea (FK)		
Cardinalidad 1:N		
Redundancia de datos		

Preguntas de reflexión — Actividad 1

Pregunta 1:

¿Qué es la normalización y qué tres problemas concretos resuelve en la panadería Dulce Migaja?

Respuesta:

Pregunta 2:

¿Cuál es la diferencia entre una clave primaria (PK) y una clave foránea (FK)? Da un ejemplo con dos tablas de Dulce Migaja.

Respuesta:

Pregunta 3:

En la tabla ventas_original, el nombre del cliente se repite en cada fila de venta. ¿Qué forma normal se viola? ¿Qué ocurre si el cliente cambia su número de teléfono?

Respuesta:

Pregunta 4:

¿Por qué en el modelo normalizado la tabla ventas solo guarda id_cliente en lugar del nombre completo del cliente?

Respuesta:

Modelo Entidad-Relación de Dulce Migaja

Observa el siguiente modelo ER ya normalizado. Úsalo como referencia para las actividades de SQL.

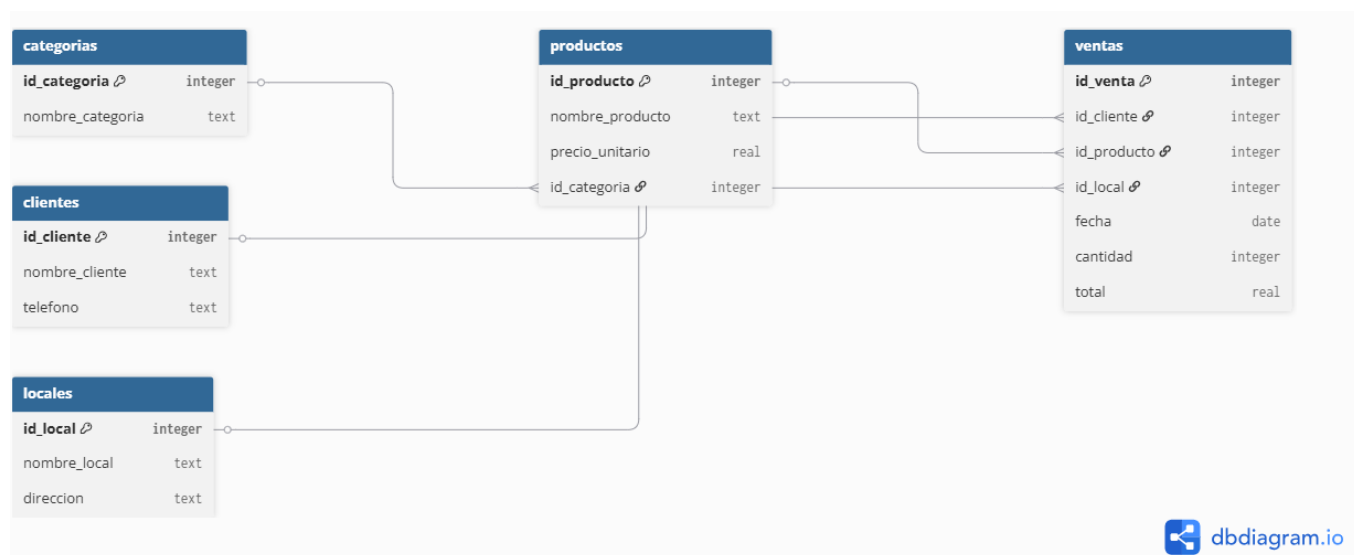
Entidad	Atributos principales	PK	Relación
categorias	id_categoria, nombre	id_categoria	1:N con productos
productos	id_producto, nombre, id_categoria (FK), precio_unitario	id_producto	N:1 categorias; 1:N ventas
clientes	id_cliente, nombre, distrito	id_cliente	1:N con ventas
locales	id_local, nombre, distrito	id_local	1:N con ventas
ventas	id_venta, id_cliente(FK), id_producto(FK), id_local(FK), fecha, cantidad, total	id_venta	N:1 clientes, productos, locales

Actividad 2: Exploración con SQLite

Antes de ejecutar las consultas, observa el diagrama en el siguiente enlace

<https://dbdiagram.io/d/6a15edf4b62396d22c76ca64>

la tabla ventas solo contiene IDs (claves foráneas) que apuntan a clientes, productos y locales. Gracias a esta estructura, cada dato se guarda una sola vez. El comando JOIN es el que permite "reunir" esa información cuando se necesita un reporte completo.



Paso 0 — Descargar la base de datos desde GitHub y conectarse

Ejecuta este bloque primero. Descarga automáticamente la base de datos de Dulce Migaja desde el repositorio GitHub y abre la conexión con SQLite. Explicación línea por línea:

- `curl` descarga el archivo `.db` desde la URL de GitHub y lo guarda localmente; si el entorno no acepta `--ssl-no-revoke`, se reintenta con `curl` normal.
- `sqlite3.connect`: abre la conexión con la base de datos descargada.
- `pd.read_sql_query`: ejecuta consultas SQL y devuelve los resultados como tabla pandas.

```
import sqlite3
import pandas as pd
import subprocess
from pathlib import Path

# Descargar la base de datos desde GitHub
url = (
    'https://raw.githubusercontent.com/Rociosayan/'
    'PMD1_Fundamentos_Gestion_Datos/main/casos/'
    '17_panaderia_normalizacion/panaderia_normalizacion.db'
)
db_path = Path('panaderia_normalizacion.db')

# Se usa curl para evitar problemas de certificados SSL en algunas
# instalaciones locales de Python.
comando = ['curl', '--ssl-no-revoke', '-L', '--fail', '-o', str(db_path), url]
resultado = subprocess.run(comando, capture_output=True, text=True)

# En algunos entornos Linux/Colab, curl puede no necesitar --ssl-no-revoke.
if resultado.returncode != 0:
```

```

comando = ['curl', '-L', '--fail', '-o', str(db_path), url]
resultado = subprocess.run(comando, capture_output=True, text=True)

if resultado.returncode != 0:
    raise RuntimeError('No se pudo descargar la base de datos desde GitHub.
Revisa tu conexi?n a Internet.')

if not db_path.exists() or db_path.stat().st_size == 0:
    raise RuntimeError('La base de datos se descarg? vac?a. Vuelve a ejecutar
la celda.')

# Conectarse a la base de datos SQLite
conn = sqlite3.connect(db_path)
print('Base de datos de Dulce Migaja cargada correctamente.')

```

Paso 1 — La tabla original: así guardaba Dulce Migaja sus ventas

Antes de la normalización, todos los datos de cada venta estaban en una sola tabla. Observa qué columnas tiene y qué información se repite.

```

# Ver las tablas disponibles en la base de datos
tablas = pd.read_sql_query(
    '''
    SELECT name AS tabla
    FROM sqlite_schema
    WHERE type = 'table'
    AND name NOT LIKE 'sqlite_%'
    ORDER BY name
    ''',
    conn
)

display(tablas)

# Tabla original desnormalizada
pd.read_sql_query('SELECT * FROM ventas_original LIMIT 5', conn)

# Tablas normalizadas: categorias y productos
display(pd.read_sql_query('SELECT * FROM categorias', conn))
display(pd.read_sql_query('SELECT * FROM productos LIMIT 5', conn))

# Tablas normalizadas: clientes, locales y ventas
display(pd.read_sql_query('SELECT * FROM clientes LIMIT 5', conn))
display(pd.read_sql_query('SELECT * FROM locales', conn))
display(pd.read_sql_query('SELECT * FROM ventas LIMIT 5', conn))

# Ver los primeros registros de la tabla original desnormalizada
df_original = pd.read_sql_query(
    'SELECT * FROM ventas_original LIMIT 15',
    conn
)
display(df_original)

```

Pregunta 1:

¿Qué columnas de `ventas_original` se repiten en varias filas? Menciona dos ejemplos concretos y explica qué problema genera esa repetición para el negocio.

Respuesta:

Paso 2 — ¿Cuántas veces se repite la información?

Contamos cuántas veces aparece cada cliente y categoría. Esto muestra cuántos registros habría que actualizar si cambia un dato.

```
# Cuantas veces se repite cada cliente en la tabla original
df_rep_cli = pd.read_sql_query(
    '''
        SELECT cliente_nombre, COUNT(*) AS veces_repetido
        FROM ventas_original
        GROUP BY cliente_nombre
        ORDER BY veces_repetido DESC
        LIMIT 10
    ''',
    conn
)
display(df_rep_cli)

# Cuantas veces se repite cada categoria
df_rep_cat = pd.read_sql_query(
    '''
        SELECT categoria_nombre, COUNT(*) AS veces_repetido
        FROM ventas_original
        GROUP BY categoria_nombre
        ORDER BY veces_repetido DESC
    ''',
    conn
)
display(df_rep_cat)
```

Pregunta 2:

¿Cuántas veces aparece el cliente más repetido? Si ese cliente cambiara su teléfono, ¿cuántos registros tendría que actualizar el administrador en la tabla original? ¿Qué soluciona la normalización?

Respuesta:

Paso 3 — Tablas normalizadas: categorías y productos

Cada categoría se guarda una sola vez (1FN). La tabla productos usa id_categoria como clave foránea en lugar de repetir el texto (3FN — elimina dependencia transitiva).

```
# Tabla de categorias: cada categoria una sola vez
df_cat = pd.read_sql_query('SELECT * FROM categorias', conn)
display(df_cat)
```

```
# Tabla de productos: usa id_categoria como FK en vez de texto repetido
df_prod = pd.read_sql_query('SELECT * FROM productos', conn)
display(df_prod)
```


Pregunta 3:

¿Cuántas categorías distintas existen? ¿Qué columna de productos actúa como clave foránea? Si cambia el nombre de una categoría, ¿en cuántos lugares habría que actualizarlo con este diseño normalizado?

Respuesta:

Paso 4 — Clientes, locales y la tabla ventas normalizada

La tabla ventas ya no repite nombres: solo guarda id_cliente, id_producto e id_local. Todos los atributos dependen completamente de id_venta (clave primaria) → cumple 2FN.

```
# Tabla de clientes
df_cli = pd.read_sql_query('SELECT * FROM clientes', conn)
display(df_cli)
```

```
# Tabla de locales
df_loc = pd.read_sql_query('SELECT * FROM locales', conn)
display(df_loc)
```

```
# Tabla de ventas normalizada
df_ven = pd.read_sql_query('SELECT * FROM ventas LIMIT 10', conn)
display(df_ven)
```

Pregunta 4:

¿Qué diferencia principal observas entre ventas_original y ventas? ¿Por qué la tabla ventas normalizada solo contiene números (IDs) en lugar de nombres? ¿Qué forma normal garantiza esto?

Respuesta:

Paso 5 — ¿Qué producto tiene el precio más alto?

```
# Productos ordenados de mayor a menor precio
df_precio = pd.read_sql_query(
    '''
    SELECT nombre AS producto, precio_unitario
    FROM productos
    ORDER BY precio_unitario DESC
    ''',
    conn
)
display(df_precio)
```

Pregunta 5:

¿Qué producto tiene el mayor precio unitario? ¿Y el menor? ¿Qué diferencia hay entre ambos precios?

Respuesta:

Paso 6 — ¿Cuántas ventas tiene cada categoría?

```
# Ventas totales por categoria (usando la tabla original para ver el dato directo)
df_cat_ven = pd.read_sql_query(
    '''
        SELECT categoria_nombre, COUNT(*) AS total_ventas
        FROM ventas_original
        GROUP BY categoria_nombre
        ORDER BY total_ventas DESC
    ''',
    conn
)
display(df_cat_ven)
```

Pregunta 6:

¿Qué categoría tiene más ventas registradas? ¿Qué recomendarías al equipo de producción basándote en este resultado?

Respuesta:

Paso 7 — ¿Qué local registra más ventas? (JOIN)

Para obtener el nombre del local en lugar del ID, se usa INNER JOIN que conecta ventas con locales mediante la clave foránea id_local.

```
# Ventas por local: se usa JOIN para obtener el nombre del local
df_loc_ven = pd.read_sql_query(
    '''
        SELECT l.nombre AS local, COUNT(*) AS total_ventas
        FROM ventas v
        INNER JOIN locales l ON v.id_local = l.id_local
        GROUP BY l.id_local, l.nombre
        ORDER BY total_ventas DESC
    ''',
    conn
)
display(df_loc_ven)
```

Pregunta 7:

¿Qué local registra más ventas? ¿Por qué fue necesario usar JOIN en lugar de consultar solo la tabla ventas?

Respuesta:

Paso 8 — Reporte completo con JOIN

Para generar un reporte legible con nombres en lugar de IDs, combinamos las cuatro tablas usando INNER JOIN. Cada JOIN conecta ventas con una tabla diferente usando su clave foránea.

```
# Reporte completo: nombre del cliente, producto, local y monto
df_reporte = pd.read_sql_query(
    '''
    SELECT
        v.id_venta,
        c.nombre AS cliente,
        p.nombre AS producto,
        l.nombre AS local,
        v.cantidad,
        p.precio_unitario,
        ROUND(v.cantidad * p.precio_unitario, 2) AS total_venta
    FROM ventas v
    INNER JOIN clientes c ON v.id_cliente = c.id_cliente
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN locales l ON v.id_local = l.id_local
    ORDER BY total_venta DESC
    LIMIT 12
    ''',
    conn
)
display(df_reporte)
```

Pregunta 8:

¿Por qué fue necesario usar tres JOIN para generar este reporte? ¿Qué ventaja tiene sobre guardar todo en ventas_original?

Respuesta:

Paso 9 — ¿Qué cliente compra con más frecuencia?

```
# Top 5 clientes con mas compras
df_top_cli = pd.read_sql_query(
    '''
    SELECT c.nombre AS cliente, COUNT(*) AS total_compras
    FROM ventas v
    INNER JOIN clientes c ON v.id_cliente = c.id_cliente
    GROUP BY c.id_cliente, c.nombre
    ORDER BY total_compras DESC
    LIMIT 5
    ''',
    conn
)
display(df_top_cli)
```

Pregunta 9:

¿Quién es el cliente más frecuente? ¿Qué acción concreta de marketing podría implementar Dulce Migaja para retenerlo?

Respuesta:

Paso 10 — ¿Qué producto genera más ingresos?

```
# Ingresos totales por producto
df_ingresos = pd.read_sql_query(
```

```
'''
SELECT
    p.nombre AS producto,
    COUNT(*) AS veces_vendido,
    SUM(v.cantidad) AS unidades_totales,
    ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS ingresos_totales
FROM ventas v
INNER JOIN productos p ON v.id_producto = p.id_producto
GROUP BY p.id_producto, p.nombre
ORDER BY ingresos_totales DESC
'''
conn
)
display(df_ingresos)
```

Pregunta 10:

¿Qué producto genera más ingresos totales? ¿Coincide con el producto más caro? ¿Qué conclusión puedes extraer sobre precio vs. volumen de ventas?

Respuesta:

Actividad 3: Aplicación y Reflexión

Responde las siguientes situaciones sin ejecutar código adicional. Usa tu comprensión del modelo normalizado y los resultados de la Actividad 2. Escribe respuestas en párrafos completos.

Situación 1:

El dueño decide que cada producto puede pertenecer a más de una categoría. Por ejemplo, "Pan de chocolate" podría ser "Panadería" y "Postres" al mismo tiempo.

Pregunta 1:

¿La estructura actual de las tablas productos y categorías soporta este cambio? ¿Qué forma normal se vería afectada? ¿Qué tabla nueva habría que crear?

Respuesta:

Situación 2:

Un desarrollador propone agregar las columnas nombre_cliente, telefono_cliente y direccion_cliente directamente a la tabla ventas para evitar el JOIN.

Pregunta 2:

¿Qué forma normal se violaría con esta decisión? ¿Qué problemas concretos generaría para Dulce Migaja cuando un cliente cambie su dirección?

Respuesta:

Situación 3:

La panadería quiere ofrecer delivery. Necesita registrar: dirección de entrega, hora estimada, nombre del repartidor y costo del envío.

Pregunta 3:

¿La estructura actual soporta este requerimiento? ¿Qué tabla o tablas nuevas crearías? Describe brevemente las columnas de cada tabla nueva.

Respuesta:

Situación 4:

Imagina que en la tabla productos hubiera una columna nombre_categoria de tipo texto (como en ventas_original) en vez de id_categoria (FK).

Pregunta 4:

¿Qué dependencia transitiva existiría? ¿Qué forma normal se violaría? ¿Qué pasaría si se cambia el nombre de una categoría?

Respuesta:

Situación 5 — Integración:

Pregunta 5:

Explica con tus propias palabras por qué Dulce Migaja tomó la decisión correcta al normalizar su base de datos. Menciona al menos tres beneficios concretos que obtiene el negocio frente a la tabla ventas_original.

Respuesta:

Conclusiones

Escribe cinco conclusiones sobre lo aprendido en este laboratorio:

1. ¿Qué problema concreto resuelve la 1FN, la 2FN y la 3FN?

R: _____

2. ¿Qué diferencia fundamental existe entre ventas_original y el diseño normalizado?

R: _____

3. ¿Para qué sirve el JOIN y cuándo lo usarías en el negocio de Dulce Migaja?

R: _____

4. ¿Qué consulta SQL de la Actividad 2 fue más útil para la toma de decisiones?

R: _____

5. ¿Por qué es más fácil mantener y actualizar una base de datos normalizada?

R: _____

Material Complementario

- Coronel, C. & Morris, S. (2019). Database Systems: Design, Implementation & Management. Cengage. Capítulos 4 y 5.
- Documentación oficial SQLite: <https://www.sqlite.org/docs.html>
- Repositorio del curso — base de datos panadería:
https://github.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos
- Diapositivas Semana 4: Modelo Entidad-Relación, cardinalidad y normalización — disponibles en el aula virtual.

Código para el diagrama en <https://dbdiagram.io/d/6a15edf4b62396d22c76ca64>

```
Table categorias {  
  id_categoria integer [pk]  
  nombre_categoria text  
}  
  
Table productos {  
  id_producto integer [pk]  
  nombre_producto text  
  precio_unitario real  
  id_categoria integer [ref: > categorias.id_categoria]  
}  
  
Table clientes {  
  id_cliente integer [pk]  
  nombre_cliente text  
  telefono text  
}  
  
Table locales {  
  id_local integer [pk]  
  nombre_local text  
  direccion text  
}  
  
Table ventas {  
  id_venta integer [pk]  
  id_cliente integer [ref: > clientes.id_cliente]  
  id_producto integer [ref: > productos.id_producto]  
  id_local integer [ref: > locales.id_local]  
  fecha date  
  cantidad integer  
  total real  
}
```

Rúbrica Holística de Evaluación

CURSO:	Fundamentos de Gestión de Datos	SEMESTRE / PERIODO:	2026-I
ACTIVIDAD:	Laboratorio Dirigido N° 04	FECHA / SECCIÓN:	_____ / _____
DOCENTE:	Pilar Rocío Sayán Mejía	ESTUDIANTE:	_____

Criterio de evaluación	Excelente	Bueno	Requiere mejora	No aceptable	Puntaje logrado
Revisión de conceptos (Actividad 1) /5 pts	Completa todos los conceptos con definiciones propias y ejemplos precisos del caso.	Completa la mayoría de conceptos con definiciones aceptables.	Completa parcialmente o copia textualmente del material.	No completa la tabla de conceptos.	
Identificación de redundancias /4 pts	Identifica correctamente todas las columnas que se repiten y explica el problema de negocio.	Identifica las columnas repetidas pero la explicación del problema es incompleta.	Identifica solo algunas columnas repetidas o la explicación es incorrecta.	No identifica redundancias.	
Comprensión de tablas normalizadas /5 pts	Explica correctamente 1FN, 2FN y 3FN usando ejemplos del caso Dulce Migaja.	Explica correctamente dos de las tres formas normales.	Solo explica una forma normal o los ejemplos no corresponden al caso.	No comprende las formas normales.	
Consultas SQL y resultados /5 pts	Ejecuta todas las consultas, los resultados son correctos y las respuestas interpretan el negocio.	Ejecuta la mayoría de consultas con resultados correctos, pero interpretación incompleta.	Ejecuta las consultas, pero no interpreta los resultados del negocio.	No ejecuta las consultas o los resultados son incorrectos.	
Análisis reflexivo (Actividad 3) /5 pts	Responde todas las situaciones con razonamiento sólido,	Responde la mayoría de situaciones con razonamiento aceptable.	Responde parcialmente o el razonamiento es superficial.	No responde las situaciones de análisis.	

Criterio de evaluación	Excelente	Bueno	Requiere mejora	No aceptable	Puntaje logrado
	conectando teoría y negocio.				
Conclusiones /3 pts	Cinco conclusiones propias, concretas y relacionadas con el caso Dulce Migaja.	Tres o cuatro conclusiones propias y pertinentes.	Conclusiones genéricas o copiadas del material.	No escribe conclusiones o son irrelevantes.	
Puntualidad en la entrega /1 pt	Entrega en el plazo establecido.			Entrega fuera del plazo.	
TOTAL					/20 pts

Acciones a cumplir	Puntaje
Puntualidad y dedicación	1
Cumplimiento de tiempos establecidos	1
Conclusiones: ortografía y redacción	1
Puntaje total	

Comentarios respecto del desempeño del alumno

Nivel	Descripción
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo la mayoría de requerimientos.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad cumpliendo pocos requerimientos.
No aceptable	No demuestra entendimiento del problema o actividad.

